ASSIGNMENT 2 - CO2

Functions and Object-Oriented Concepts in C++

Answer Report | Total Marks: 50 | Submission Date: 31 October 2026

Student details	Fill before submission
Name	Nithya shree K
Register number	192521145
Course code / Name	DSA0103 OOPS with C++
Faculty	Arun Kumar
GitHub repository	https://github.com/192521145simats-uxl/DSA0103-OOPS-with-C

Contents

Q1. Call by value, reference and pointer (6 marks)

Q2. Return by reference and its dangers (4 marks)

Q3. Default arguments and overloading (6 marks)

Q4. Savings account with a transaction array (6 marks)

Q5. Static members and arrays of objects (7 marks)

Q6. Friend and inline functions (8 marks)

Q7. Diagnose, redesign and validate (13 marks)

Execution and GitHub submission checklist

Execution record

The outputs labelled Expected are predictions derived from C++ rules. The accompanying local verification used MinGW-w64 GCC 8.1.0 with C++17 on Windows, and all 12 source files compiled and ran successfully. Their observed outputs matched the expected results. The Q2 warning is intentional. Record your own predictions before your individual execution; the local checks do not replace the required student execution or screenshots.

Language standard: C++17. Suggested compilation: g++ -std=c++17 -Wall -Wextra program.cpp -o program.exe. Run on Windows using .\program.exe. All examples use fixed test data and require no input file.

Q1. Call by Value, Reference and Pointer

1. Three implementations and prediction

Use a fresh fee of 45000 for each test. A 10% concession is 4500. Before execution, the stored fees are predicted to be 45000, 40500 and 40500 for value, reference and pointer respectively.

```
#include <iostream>
void byValue(long long fee, int percent);
void byReference(long long& fee, int percent);
void byPointer(long long* fee, int percent);
long long newFee(long long fee, int percent);
int main() {
   long long a=45000, b=45000, c=45000;
   byValue(a,10); byReference(b,10); byPointer(&c,10);
   std::cout << a << ' ' << b << ' ' << c << '\n';
   long long fee=45000;
   for (int rate : {10,15,25}) fee=newFee(fee,rate);
   std::cout << "Sequential fee: " << fee << '\n';
}
void byValue(long long fee, int percent) {
   fee -= fee*percent/100;
}
void byReference(long long& fee, int percent) {
   fee -= fee*percent/100;
}
void byPointer(long long* fee, int percent) {
   if (fee) *fee -= *fee*percent/100;
}
long long newFee(long long fee, int percent) {
   return fee-fee*percent/100;
}
```

Expected:
45000 40500 40500
Sequential fee: 25819

The prototypes declare each interface before main. By value creates an independent parameter object; changing it does not change a. A reference aliases b. A pointer parameter is a copy of the address, but dereferencing it accesses c. The language does not require these parameters to occupy particular stack locations: optimisation may use registers or remove them.

Use const Student& when a function only reads a large Student object. It avoids copying and prevents modification through that reference. For a small long long, value is usually simpler and inexpensive. A pointer is useful when no object is a valid possibility; check null before dereferencing.

2. Interface decision

Prefer returning the updated fee: fee = newFee(fee, rate). It makes changes explicit, supports previewing a concession and composes cleanly. For sequential rates, 45000 -> 40500 -> 34425 -> 25819. Integer division truncates each discount. This differs from adding the percentages: 50% off the original gives 22500. The hostel must specify its concession-combination policy. Rates should be validated in 0..100, and production arithmetic must prevent multiplication overflow.

3. Supplied program: predicted output

1 12 13
1 100
16 13 6

The final line has one trailing space before its newline. change copies a, so the caller keeps a=1; b and c become 12 and 13. bigger(a,c) returns a reference to c, so assignment sets c to 100. show(1) uses y=5 and z=10; show(1,2) uses only z=10; show(1,2,3) uses no defaults. The predicted and expected outputs agree.

```
#include <iostream>
void change(int a, int& b, int* c) {
   a+=10; b+=10; *c+=10;
}
int& bigger(int& p, int& q) { return p>q ? p:q; }
void show(int x,int y=5,int z=10) {
   std::cout << x+y+z << ' ';
}
int main() {
   int a=1,b=2,c=3;
   change(a,b,&c);
   std::cout << a << ' ' << b << ' ' << c << '\n';
   bigger(a,c)=100;
   std::cout << a << ' ' << c << '\n';
   show(1); show(1,2); show(1,2,3); std::cout << '\n';
}
```

Observed local output: exactly the predicted three lines, including the trailing space. No differences. Individual student execution record: _____

4. Complete the Code

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
   fee -= fee * percent / 100;
}
void applyConcession(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}
int main()
{
   long long hostelFee = 45000;
   applyByValue(hostelFee, 10);
   std::cout << "after call by value : " << hostelFee << '\n';
   applyConcession(hostelFee, 10);
   std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Expected:
after call by value : 45000
after call by reference: 40500

The first function changes only its local copy. The reference version changes the original hostelFee.

5. Alternative and complexity

Parameter / result	Time per call	Copy / auxiliary space
long long by value; return long long	O(1)	One scalar parameter and scalar result; O(1)
long long& / const long long&	O(1)	No fee object copy; O(1) binding
long long*	O(1)	One pointer value; O(1)
Student by value, size S	O(S) for a deep copy	O(S) copied storage
Student& / const Student&	O(1) binding	No object copy; O(1)

Fee arithmetic is O(1) in every scalar version. Copying a large object is not automatically O(1); the copy constructor and owned data determine its cost. A const reference is read-only through that alias but does not make the entire object globally immutable.

Q2. Return by Reference and Its Dangers

1. Safe functions and a deliberately unsafe function

```
#include <iostream>
#include <stdexcept>
int& highest(int* a, int n) {
    if (!a || n<=0) throw std::invalid_argument("empty marks");
    int best=0;
    for (int i=1;i<n;++i) if (a[i]>a[best]) best=i;
    return a[best];
}
const int& highestReadOnly(const int* a, int n) {
    if (!a || n<=0) throw std::invalid_argument("empty marks");
    int best=0;
    for (int i=1;i<n;++i) if (a[i]>a[best]) best=i;
    return a[best];
}
int& badReference() {
    int local=99;
    return local; // Deliberate error: dangling reference.
}
int highestIndex(const int* a,int n) {
    if (!a || n<=0) throw std::invalid_argument("empty marks");
    int best=0;
    for (int i=1;i<n;++i) if (a[i]>a[best]) best=i;
    return best;
}
int* highestPointer(int* a,int n) {
    return (!a || n<=0) ? nullptr : &a[highestIndex(a,n)];
}
int main() {
    int marks[]={70,88,81};
    highest(marks,3)+=2;
    std::cout << marks[1] << '\n';
    highest(marks,3)=95;
    std::cout << highestReadOnly(marks,3) << '\n';
    int copy=highestReadOnly(marks,3);
    copy+=2;
    std::cout << copy << ' ' << marks[1] << '\n';
    int i=highestIndex(marks,3); marks[i]+=2;
    int* ptr=highestPointer(marks,3); if(ptr) *ptr+=2;
    std::cout << marks[1] << '\n';
    // Never read or write through badReference().
}
```

Expected:
90
95
97 95
99

A returned int& is an alias to a surviving array element and can be an assignment target. A returned const int& avoids copying but prohibits assignment through it. Returning int creates a separate scalar value. Neither kind of reference owns the object or extends the lifetime of an ordinary local variable.

2. Dangling-reference diagnosis

local is destroyed when badReference returns. Any later access through the returned reference has undefined behaviour: no fixed output, crash or value is guaranteed. Do not demonstrate this by relying on a crash. Compile the function with warnings and keep its call disabled. Typical GCC wording is "reference to local variable ... returned" (-Wreturn-local-addr); exact wording depends on compiler/version.

Observed GCC 8.1.0 diagnostic: warning: reference to local variable 'local' returned [-Wreturn-local-addr]. The unsafe function was compiled but never called.

3. Changed condition: growing vector

If push_back increases vector capacity, reallocation moves elements to new storage and invalidates all existing element references and pointers. If capacity does not change, existing element references survive push_back. reserve is useful only while its capacity is not exceeded. Erasing or inserting can invalidate references and can change which element an index denotes.

```
// With #include <vector>, use an index across append-only growth:
std::vector<int> marks{70,88,81};
int index=highestIndex(marks.data(),static_cast<int>(marks.size()));
marks.push_back(92);
marks.at(index)+=2; // Still the old element; not necessarily highest now.
// Recompute highestIndex if the current maximum is required.
```

4. Index and pointer alternatives

Index: int i=highestIndex(a,n); a[i]+=2. Pointer: int* p=highestPointer(a,n); if(p) *p+=2. Index, reference and pointer each need an O(n) scan and O(1) extra space. Reference and pointer both dangle after vector reallocation. An index remains meaningful after append-only reallocation when its original element still exists, but it must be bounds-checked and may no longer identify the maximum. Choose an index for append-only growth; recompute after edits that change positions or maximum value. Ties in all implementations select the first maximum.

Q3. Default Arguments and Function Overloading

1. Why the original declarations are ambiguous

charge(double), charge(double,bool), and charge(double,bool=false,bool=false) have overlapping viable calls. charge(2.0) can match both the first and third with the same conversion quality. charge(2.0,true) can match both the second and third. The number of omitted default arguments does not select a winner. Overloading solely by return type is also invalid.

2. Redesigned functions

Use exactly one overload per input kind: int means grams, double means kilograms and Parcel holds its own options. The illustrative tariff is Rs 50 per kg, plus Rs 40 for express and Rs 20 for insurance. Negative weights are rejected; fractional weights are charged proportionally.

```
#include <iostream>
#include <iomanip>
#include <stdexcept>
struct Parcel { double kg; bool express=false; bool insured=false; };
double charge(double kg,bool express=false,bool insured=false) {
   if(kg<0) throw std::invalid_argument("negative weight");
   return kg*50+(express?40:0)+(insured?20:0);
}
double charge(int grams,bool express=false,bool insured=false) {
   if(grams<0) throw std::invalid_argument("negative weight");
   return charge(grams/1000.0,express,insured);
}
double charge(const Parcel& p) {
   return charge(p.kg,p.express,p.insured);
}
int main() {
   std::cout << std::fixed << std::setprecision(2);
   std::cout << charge(2) << '\n';
   std::cout << charge(2.0f) << '\n';
   std::cout << charge(2.0) << '\n';
   std::cout << charge(2000,true) << '\n';
   std::cout << charge(2.0,true,true) << '\n';
   std::cout << charge(Parcel{2.0,false,true}) << '\n';
}
```

Call	Predicted selection	Expected charge
charge(2)	int overload: exact match	0.10
charge(2.0f)	double overload: promotion	100.00
charge(2.0)	double overload: exact match	100.00
charge(2000,true)	int overload; insured=false	140.00
charge(2.0,true,true)	double overload; no defaults	160.00
charge(Parcel{2.0,false,true})	Parcel overload	120.00

Observed local console output matched all six charges in the table, one per line. No discrepancies were found. Individual student execution record: _____

3. Selection rules and defaults

For these calls the useful ranking is exact match, then promotion, then conversion. float to double is a promotion; float to int is a conversion, so charge(2.0f) selects double. Other overload sets can still be ambiguous, for example a long argument convertible to both int and double without a better match. The guarantee here covers the stated input types, not every possible C++ type.

Defaults must be available for trailing parameters after a parameter with a default, so omitted positional arguments have a clear meaning. For example charge(2.0,true) omits insured, while charge(2.0,,true) is invalid. In practice place defaults in one visible declaration and do not repeat them in its definition.

4. Alternative: a single Parcel interface

```
double chargeParcel(const Parcel& p) {
   if(p.kg<0) throw std::invalid_argument("negative weight");
   return p.kg*50+(p.express?40:0)+(p.insured?20:0);
}
// chargeParcel(Parcel{2.0});
// chargeParcel(Parcel{2.0,true,true});
```

A structure centralises new options: add a field with a default and update the calculation. A template can accept arithmetic weights, but must encode units explicitly; a generic numeric type alone does not distinguish grams from kg. The structure design is preferable as options grow, while overloads are convenient at the input boundary.

Overload resolution occurs at compile time and adds no run-time dispatch cost. Each tariff calculation is O(1). Scalar arguments and a fixed Parcel by value use O(1) space; const Parcel& copies no Parcel. If Parcel later owns variable-size data, passing it by value may require O(S) copying. A function template is instantiated at compile time and may increase generated code size.

Q4. Savings Account Mini-Statement

1. Original design: shift a private fixed array

Transactions store signed amounts: deposits are positive and withdrawals negative. Invalid deposits and withdrawals are rejected and not recorded. The balance and transaction array remain private so all changes pass through validation.

```
#include <iostream>
#include <iomanip>
#include <string>
class SavingsAccount {
   static constexpr int LIMIT=10;
   std::string number;
   double balance;
   double transactions[LIMIT]{};
   int count=0;
   void record(double amount);
public:
   explicit SavingsAccount(const std::string& n)
      : number(n),balance(0) {}
   bool deposit(double amount);
   bool withdraw(double amount);
   double getBalance() const { return balance; }
   void miniStatement() const;
};
void SavingsAccount::record(double amount) {
   if(count<LIMIT) transactions[count++]=amount;
   else {
      for(int i=1;i<LIMIT;++i)
         transactions[i-1]=transactions[i];
      transactions[LIMIT-1]=amount;
   }
}
bool SavingsAccount::deposit(double amount) {
   if(amount<=0) return false;
   balance+=amount; record(amount); return true;
}
bool SavingsAccount::withdraw(double amount) {
   if(amount<=0 || amount>balance) return false;
   balance-=amount; record(-amount); return true;
}
void SavingsAccount::miniStatement() const {
   std::cout << std::fixed << std::setprecision(2);
   std::cout << "Account: " << number << '\n';
   for(int i=0;i<count;++i) std::cout << transactions[i] << '\n';
   std::cout << "Balance: " << balance << '\n';
}
int main() {
   SavingsAccount a("SA101");
   for(int i=1;i<=11;++i) a.deposit(i*100);
   a.miniStatement();
   std::cout << std::boolalpha
             << "Excess withdrawal accepted: " << a.withdraw(7000)
```

```
        << '\n';
}
```

Expected:
Account: SA101
200.00
300.00
400.00
500.00
600.00
700.00
800.00
900.00
1000.00
1100.00
Balance: 6600.00
Excess withdrawal accepted: false

The constructor and getter are defined inside the class; deposit, withdraw, record and miniStatement are defined outside with SavingsAccount::. The 11th transaction discards 100, shifts nine entries left and appends 1100. Removing a statement entry never removes its financial effect from balance.

2. Alternatives and the last-50 requirement

Design	Record when full	Read statement	Storage
Shifted array of capacity k	O(k), k assignments	O(k)	O(k), fixed contiguous storage
Circular array	O(1), one assignment	O(k) in logical order	O(k) plus two indices
std::deque	O(1) end insertion/removal	O(k)	O(k) with block/allocation overhead

```
// Circular buffer record operation:
if(count<capacity) {
   data[(start+count)%capacity]=amount;
   ++count;
} else {
   data[start]=amount;
   start=(start+1)%capacity;
}
// Print data[(start+i)%capacity] for i in [0,count).

// Deque alternative, with #include <deque>:
// std::deque<double> history;
// if(history.size()==capacity) history.pop_front();
// history.push_back(amount);
```

Choose a circular array with a named capacity or a capacity template parameter. Changing 10 to 50 changes one configuration value and preserves O(1) recording. A shifted array also needs only a capacity change but its full-buffer work rises from 10 to 50 assignments. With a literally fixed capacity of 10 both are O(1) in total number of transactions; O(k) describes how cost scales with history capacity. For real banking, store integer paise and validate finite inputs rather than relying on binary floating-point money.

Q5. Static Members and Arrays of Objects

1. Class design and Complete the Code

issued belongs to the bus-wide allocation state, while name and seat belong to one ticket. CAPACITY is a class constant. issued is declared in the class and defined once outside it. count() is static because no particular ticket is needed.

```
#include <iostream>
#include <string>
class BusTicket {
   static int issued;
   static const int CAPACITY = 3;
   int seat;
   std::string name;
public:
   explicit BusTicket(const std::string& n) : seat(0), name(n)
   {
      if (issued < CAPACITY) seat = ++issued;
   }
   static int count()
   {
      return issued;
   }
   void print() const
   {
      std::cout << name << ": "
        << (seat ? "Seat " + std::to_string(seat)
                 : std::string("Waiting")) << '\n';
   }
};
int BusTicket::issued = 0;
int main()
{
   BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                        BusTicket("Chitra"), BusTicket("Dev")};
   for (const BusTicket& t : trip) t.print();
   std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Expected:
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3

The array explicitly constructs all four elements because this class has no default constructor. The fourth object still exists but has seat=0. A static function has no this pointer, so it cannot directly read name or seat; it could read them through an explicitly supplied BusTicket object. An ordinary static data member requires one definition; C++17 inline static is an alternative. This in-class integral capacity constant needs no separate definition in these value-only uses.

2. Supplied Token program: prediction

```
#include <iostream>
class Token {
   static int next;
   int id;
public:
   Token() { id=++next; }
   int get() const { return id; }
   static int issued() { return next-100; }
};
int Token::next=100;
int main() {
   Token counter[3]; Token extra;
   std::cout << counter[0].get() << ' ' << counter[2].get()
            << ' ' << extra.get() << ' ' << Token::issued() << '\n';
}
```

Predicted / expected:
101 103 104 4

The static definition sets next to 100 before main. Three array elements and extra invoke the constructor four times. The IDs are 101,102,103,104; issued subtracts the initial offset. Observed output: 101 103 104 4. No difference from prediction.

3. Function called twice

In a separate copy of complete.cpp, keep the same class and static definition, and replace main with the following:

```
void createPair() {
   BusTicket pair[2]={BusTicket("Asha"),BusTicket("Bala")};
   for(const auto& t:pair) t.print();
   std::cout << "Issued: " << BusTicket::count() << '\n';
}
int main() { createPair(); createPair(); }
```

Expected in a fresh process:
Asha: Seat 1
Bala: Seat 2
Issued: 2
Asha: Seat 3
Bala: Waiting
Issued: 3

Local ticket objects are destroyed after each call, but issued has static storage duration and is not reset. Loop iterations likewise share the same counter. This model counts allocated seats cumulatively, not live objects. It does not implement cancellation. Copying a ticket would copy its seat rather than reserve another seat; a production booking design should control copying and track booking identity.

4. Alternative designs and two buses

```
class TicketOffice {
   int issued=0;
   int capacity;
public:
   explicit TicketOffice(int cap):capacity(cap) {}
   int allocate() { return issued<capacity ? ++issued : 0; }
   int count() const { return issued; }
};
// TicketOffice busA(3), busB(3);
// In a redesigned ticket constructor:
// BusTicket(TicketOffice& office, const std::string& n)
//     : seat(office.allocate()), name(n) {}
```

A global counter also gives O(1) allocation but exposes shared state and can cause accidental interference. A private static counter encapsulates one shared bus, but all tickets of that class share it. A separate TicketOffice provides one counter per bus and supports independent buses. All use O(1) counter space for one bus and O(1) seat-allocation time, excluding passenger-name copying. With b buses, separate offices need O(b) counters. Prefer TicketOffice for a multi-bus system.

Q6. Friend Functions and Inline Functions

1. Complete the Code

```
#include <iostream>
class Time {
   int h, m;
public:
   Time(int hh, int mm) : h(hh), m(mm) {}
   friend int minutesBetween(const Time& a, const Time& b);
   friend bool earlier(const Time& a, const Time& b);
};
int minutesBetween(const Time& a, const Time& b)
{
   int difference=(a.h*60+a.m)-(b.h*60+b.m);
   return difference<0 ? -difference : difference;
}
bool earlier(const Time& a, const Time& b)
{
   return a.h*60+a.m < b.h*60+b.m;
}
int main()
{
   Time departure(9, 45), arrival(13, 10);
   std::cout << minutesBetween(departure, arrival) << ' '
             << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

Expected:
205 false

2. Correctness and interface design

Inputs a / b	Hand calculation	minutesBetween(a,b)	earlier(a,b)
09:45 / 13:10	790 - 585	205	true
12:05 / 11:55	725 - 715	10	false
11:55 / 12:05	725 - 715	10	true
12:00 / 12:00	720 - 720	0	false

Repeat the first program using these time pairs to verify the table. The supplied main asks earlier(arrival,departure), which is false. The model measures absolute difference within one day; 23:55 and 00:05 differ by 1430, not by a next-day travel duration of 10. Date or overnight rules require an additional model. Validate 0<=h<24 and 0<=m<60 in a production constructor.

A non-member function is a natural interface for treating two objects symmetrically: minutesBetween(a,b). Friendship is needed for this implementation to read h and m directly, but is not the only possible design. A member is called a.minutesTo(b), receives an implicit this object, and can read the private data of both Time objects. A friend has no this pointer and has access only because the class explicitly grants it. Making h and m public would let callers create invalid states and couple them to the representation.

3. Member and getter alternatives

```
// Add inside Time's public section:
int minutesTo(const Time& b) const {
   int d=(h*60+m)-(b.h*60+b.m);
   return d<0 ? -d:d;
}
bool isEarlier(const Time& b) const {
   return h*60+m < b.h*60+b.m;
}
int totalMinutes() const { return h*60+m; }
// Non-friend using getters:
// int d=a.totalMinutes()-b.totalMinutes();
// return d<0 ? -d:d;
```

All three approaches perform O(1) work and use O(1) auxiliary space. Passing const references avoids copies and prevents changes through those parameters. A totalMinutes getter exposes a stable meaning more cleanly than exposing raw storage. Keep the small symmetric friend functions for this exercise; for a larger library, a semantic getter plus free functions reduces privileged access while preserving the symmetric call style.

4. What inline means

inline does not force replacement of a call with its body. It mainly permits the same definition in multiple translation units under the one-definition rules; such a definition normally belongs in a header. Functions defined inside a class are implicitly inline in this ordinary C++17 setting. Optimisers may inline a function without the keyword, or keep an inline function as a call. At -O0 little optimisation is expected; at -O2 both forms may generate equivalent code. Larger inlined bodies can increase instruction-cache pressure.

5. Reproducible 10,000,000-call measurement

The benchmark below compares an ordinary free function, an inline friend, a member and a getter-based function. Inputs vary and a volatile checksum prevents removal of the whole loop. The loop overhead is included. This compares keyword variants; it does not force the ordinary function to remain out of line. Use repeated runs and the median, the same computer and compiler, and no console output inside the timed loop.

```
#include <iostream>
#include <chrono>
class Clock {
   int h,m;
public:
   Clock(int hh,int mm):h(hh),m(mm) {}
   friend int plain(const Clock&,const Clock&);
   friend inline int hinted(const Clock& a,const Clock& b) {
      int d=(a.h*60+a.m)-(b.h*60+b.m);
      return d<0 ? -d:d;
   }
   int member(const Clock& b) const {
      int d=(h*60+m)-(b.h*60+b.m); return d<0 ? -d:d;
   }
   int total() const { return h*60+m; }
};
int plain(const Clock& a,const Clock& b) {
   int d=(a.h*60+a.m)-(b.h*60+b.m); return d<0 ? -d:d;
}
int getters(const Clock& a,const Clock& b) {
   int d=a.total()-b.total(); return d<0 ? -d:d;
}
template<class F> void measure(const char* name,F fn) {
   volatile long long checksum=0;
   auto begin=std::chrono::steady_clock::now();
   for(int i=0;i<10000000;++i) {
      Clock a(i%24,i%60),b((i+7)%24,(i+13)%60);
      checksum+=fn(a,b);
   }
   auto end=std::chrono::steady_clock::now();
   double ms=std::chrono::duration<double,std::milli>(end-begin).count();
   std::cout << name << ": " << ms << " ms; " << checksum << '\n';
}
int main() {
   measure("ordinary",[](const Clock& a,const Clock& b){return plain(a,b);});
   measure("inline",[](const Clock& a,const Clock& b){return hinted(a,b);});
   measure("member",[](const Clock& a,const Clock& b){return a.member(b);});
   measure("getters",[](const Clock& a,const Clock& b){return getters(a,b);});
}
```

```
g++ -std=c++17 -Wall -Wextra -O0 benchmark.cpp -o bench_O0.exe
.\bench_O0.exe
g++ -std=c++17 -Wall -Wextra -O2 benchmark.cpp -o bench_O2.exe
.\bench_O2.exe
```

Variant	-O0 median (ms)	-O2 median (ms)
ordinary	67.0935	24.2667
inline	63.7605	24.0539
member	63.4274	24.2170
getters	73.8971	24.0392

Measurement environment: MinGW-w64 GCC 8.1.0, C++17, Windows x64. Five runs per optimisation level; table values are medians in milliseconds. All runs and all four variants produced checksum 5954164188. Runtime includes input construction and volatile-checksum loop overhead.

Q7. Diagnose, Redesign and Validate

A. Diagnose the limitation (2 marks)

Select Q4: a full transaction history shifts all surviving entries on every insertion. The cause is keeping the oldest transaction permanently at array index zero. With capacity k, each full insertion performs k-1 shift assignments plus one append assignment. For m insertions from empty and m>=k, history-element writes are k+(m-k)k, whereas a circular buffer writes m elements. These are analytical operation counts, not measured timings.

Same workload from empty	Original writes	Circular writes
m=100000, k=10	999910	100000
m=100000, k=50	4997550	100000
m=100000, k=1000	99001000	100000

The Q4 expected statement also establishes the required behaviour: after eleven deposits, the retained amounts are 200 through 1100. Attach your original output screenshot and measured runtime to make this diagnosis experimental as well as analytical.

B. Measurable objective (2 marks)

Retain exactly the latest k valid transactions in chronological order, preserve account balances, and reduce each full-history update to one history-element assignment independent of k. For 100000 insertions at k=50, target 100000 writes rather than 4997550. Compare elapsed time under identical -O2 settings; timing improvement is a hypothesis, not a guaranteed threshold.

C. Revised solution (4 marks)

Use encapsulation and a class template with capacity K. A private array still stores all transactions, but start identifies the oldest entry and count the number retained. The redesign uses two indices rather than one count: one extra size_t, with the same O(k) history storage. Chronological reading requires modular indexing, and capacity remains fixed per object type.

```
SavingsAccount<K>
 private: number, balance, data[K], start, count
 public : deposit(amount), withdraw(amount), miniStatement()
 record(amount):
   if count<K: data[(start+count)%K]=amount; count++
   else: data[start]=amount; start=(start+1)%K
 statement:
   for i=0..count-1: print data[(start+i)%K]
```

This is an explicit circular-buffer redesign rather than replacing the algorithm with a library queue.

D. Prediction before implementation (1 mark)

Predict identical retained histories and balances. Predict exactly one history write per accepted transaction in the redesigned buffer, versus k writes in a full shifted buffer. Expect the runtime advantage to grow with k, while modular arithmetic may make the original competitive for very small histories. Write your own prediction and timestamp before running: _____

E. Implement and validate (2 marks)

```
#include <iostream>
#include <iomanip>
#include <string>
#include <cstddef>
template<std::size_t K> class SavingsAccount {
   static_assert(K>0,"capacity must be positive");
   std::string number;
   double balance=0;
   double data[K]{};
   std::size_t start=0,count=0;
   void record(double x) {
      if(count<K) { data[(start+count)%K]=x; ++count; }
      else { data[start]=x; start=(start+1)%K; }
   }
public:
   explicit SavingsAccount(const std::string& n):number(n) {}
   bool deposit(double x) {
      if(x<=0) return false;
      balance+=x; record(x); return true;
   }
   bool withdraw(double x) {
      if(x<=0 || x>balance) return false;
      balance-=x; record(-x); return true;
   }
   double getBalance() const { return balance; }
   void miniStatement() const {
      std::cout << std::fixed << std::setprecision(2);
      std::cout << "Account: " << number << '\n';
      for(std::size_t i=0;i<count;++i)
         std::cout << data[(start+i)%K] << '\n';
      std::cout << "Balance: " << balance << '\n';
   }
};
int main() {
   SavingsAccount<10> a("SA101");
   for(int i=1;i<=11;++i) a.deposit(i*100);
   a.miniStatement();
   std::cout << std::boolalpha
             << "Excess withdrawal accepted: " << a.withdraw(7000)
             << '\n';
   // Use SavingsAccount<50> for a last-50 statement.
}
```

The expected output is exactly the Q4 output, including Balance: 6600.00 and false for the excess withdrawal. Compile both versions independently and compare their outputs.

Comparable validation tests

Test from an empty account	Expected result in both versions
Print before any transaction	No entries; balance 0
Deposit 100 once	One entry 100; balance 100
Deposit 100..1000	Ten entries; balance 5500
Deposit 100..1100	Entries 200..1100; balance 6600
Deposit 100..2500	Entries 1600..2500; balance 32500
Deposit 500; withdraw 200	Entries 500,-200; balance 300
Deposit 0, deposit -1, withdraw 1 on empty	All rejected; count 0; balance 0

Equal-workload benchmark

The following isolates history recording, compares every retained element, and records exact history writes. It uses the original shifting algorithm and the redesigned circular algorithm on the same integer sequence. It is not a measurement of complete banking or printing. Repeat runs and use the median; run correctness tests above on both full account classes as well.

```
#include <iostream>
#include <vector>
#include <chrono>
#include <stdexcept>
struct Shift {
   std::vector<int> a; std::size_t count=0;
   unsigned long long writes=0;
   explicit Shift(std::size_t k):a(k) {}
   void add(int x) {
      if(count<a.size()) { a[count++]=x; ++writes; }
      else {
         for(std::size_t i=1;i<a.size();++i) {
            a[i-1]=a[i]; ++writes;
         }
         a.back()=x; ++writes;
      }
   }
   int at(std::size_t i) const { return a[i]; }
};
struct Ring {
   std::vector<int> a; std::size_t count=0,start=0;
   unsigned long long writes=0;
   explicit Ring(std::size_t k):a(k) {}
   void add(int x) {
      if(count<a.size()) { a[(start+count)%a.size()]=x; ++count; }
      else { a[start]=x; start=(start+1)%a.size(); }
      ++writes;
   }
   int at(std::size_t i) const { return a[(start+i)%a.size()]; }
};
template<class H> double run(H& history,int m) {
   auto begin=std::chrono::steady_clock::now();
   for(int i=1;i<=m;++i) history.add(i);
   auto end=std::chrono::steady_clock::now();
   return std::chrono::duration<double,std::milli>(end-begin).count();
}
int main() {
   const int m=100000;
   for(int k : {10,50,1000}) {
      Shift original(k); Ring revised(k);
      double a=run(original,m),b=run(revised,m);
      bool same=original.count==revised.count;
      for(std::size_t i=0;i<original.count;++i)
         same=same && original.at(i)==revised.at(i);
      std::cout << "k=" << k << " same=" << std::boolalpha << same
        << " writes=" << original.writes << '/' << revised.writes
```

```
        << " ms=" << a << '/' << b << '\n';
      if(!same) throw std::runtime_error("history mismatch");
    }
}
```

The vector in this measurement harness allocates a fixed runtime capacity once and never grows; it does not replace either recording algorithm. Allocation and zero-initialisation occur before the timed region. Compiler optimisation may combine stores, so counted algorithmic writes are not a count of physical CPU memory instructions.

```
g++ -std=c++17 -Wall -Wextra -O2 benchmark.cpp -o compare.exe
.\compare.exe
```

k	Original median ms	Redesigned median ms	same=true?
10	0.1842	0.3682	true (all five runs)
50	1.1231	0.3725	true (all five runs)
1000	17.2653	0.4277	true (all five runs)

F. Defend the final design (2 marks)

The operation-count objective is met and verified by the benchmark: exactly one history write per transaction in the circular design, versus k writes per full-history insertion in the shifted design. All retained histories matched. At k=10 the original was faster; at k=50 and k=1000 the circular design was faster in the recorded median timings. Thus the redesign helps as capacity grows, while the tiny-history overhead trade-off is real. The full account tests also matched. Costs are one extra index, modular arithmetic and additional bookkeeping. Memory remains O(k), and printing remains O(k). Prefer the original for tiny, rarely updated histories and the circular design for frequent updates and larger capacities.

Execution and GitHub Submission Checklist

Repository name: DSA01-<RegisterNumber>-Assignment2. Replace the register-number placeholder with your own number. Create Task01 through Task07 folders, and keep each source file, actual output screenshot and any input file beside the relevant program. These examples use embedded input, so record "No external input file required" in the README.

Execute every program, including the completed code exercises. Record each prediction before its run. Copy the actual compiler warning for Q2. Complete the timing tables for Q6 and Q7. Fill the cover page and replace the GitHub line with a working link. Share the repository publicly or add the faculty as a collaborator if private.

Commit progressively, at least one meaningful commit per task. Existing work must be committed honestly; do not fabricate past timestamps or a development history. Upload the report and submit the GitHub link as required.

README template

DSA01-<RegisterNumber>-Assignment2
Name: <Your name>
Register number: <Your register number>
Compiler and version: <Paste g++ --version result>
Standard: C++17

Build and run each program
cd Task01
g++ -std=c++17 -Wall -Wextra concession.cpp -o concession.exe
.\concession.exe
Repeat for every .cpp in Task01 through Task07.
Build each file separately: each contains its own main().
For benchmark.cpp use the -O0 / -O2 commands in the report.

Inputs
No external input files required; tests are embedded in the sources.

Results
Task01: <actual stored-fee results and prediction comparison>
Task02: <moderation output and actual dangling-reference warning>
Task03: <six overload selections and charges>
Task04: <retained history and rejected withdrawal result>
Task05: <allocated seats, waiting list and twice-called function result>
Task06: <correctness tests and measured -O0/-O2 timings>
Task07: <matching histories, operation counts and measured timings>

Evidence
Save output screenshots and compiler messages in each task folder.

Final verification record

Item	Student confirmation
All .cpp files compiled and executed	_____
Predictions recorded before individual runs	_____
Actual outputs and screenshots added	_____
Compiler/version and exact commands recorded	_____
Q6 and Q7 measured timings entered	_____
Name, register number and GitHub link filled	_____
At least one honest progressive commit per task	_____
Repository access verified	_____

Local Verification Evidence

Verified on 29 October 2026 using MinGW-w64 GCC 8.1.0. Non-benchmark programs were compiled separately with -std=c++17 -Wall -Wextra -O2. Q6 benchmark used both -O0 and -O2. The only warning was the intentionally unsafe reference-to-local example. Every executable returned exit code 0.

Program	Observed result
Task01/complete.cpp	Compiled and ran; matched expected output
Task01/concession.cpp	Compiled and ran; matched expected output
Task01/predict.cpp	Compiled and ran; matched expected output
Task02/marks.cpp	Compiled and ran; matched expected output
Task03/courier.cpp	Compiled and ran; matched expected output
Task04/account.cpp	Compiled and ran; matched expected output
Task05/complete.cpp	Compiled and ran; matched expected output
Task05/predict.cpp	Compiled and ran; matched expected output
Task06/benchmark.cpp	Compiled and ran; five timing runs recorded
Task06/complete.cpp	Compiled and ran; matched expected output
Task07/benchmark.cpp	Compiled and ran; five timing runs recorded
Task07/redesigned.cpp	Compiled and ran; matched expected output

Q6 interpretation: compare the medians above. -O2 brought the four variants much closer in this run. Source-level inline does not guarantee a speedup; construction, checksum updates and optimisation of calls all contribute. No forced no-inline attribute was used, so this experiment does not isolate machine-level call overhead.

Q7 measurement protocol: 100000 inserts per capacity, five runs, -O2. The order was original then circular within each run; short sub-millisecond timings are sensitive to scheduling and order. Exact operation counts and matching histories provide stronger evidence than the smallest timing differences. Repeat or alternate order if making a deployment decision.

Q5: function called twice

Asha: Seat 1
Bala: Seat 2
Issued: 2
Asha: Seat 3
Bala: Waiting
Issued: 3

Q6: correctness tests

205 true
10 false
10 true
0 false

Q4 and Q7: same seven correctness tests

All seven scenarios matched exactly.

```
deposits=0
Account: TEST
Balance: 0.00
deposits=1
Account: TEST
100.00
Balance: 100.00
deposits=10
Account: TEST
100.00
200.00
300.00
400.00
500.00
600.00
700.00
800.00
900.00
1000.00
Balance: 5500.00
deposits=11
Account: TEST
200.00
300.00
400.00
500.00
600.00
700.00
800.00
900.00
1000.00
1100.00
Balance: 6600.00
deposits=25
Account: TEST
1600.00
1700.00
1800.00
1900.00
2000.00
2100.00
2200.00
2300.00
2400.00
```

2500.00
Balance: 32500.00
Account: WITHDRAW
500.00
-200.00
Balance: 300.00
rejected=true true true
Account: INVALID
Balance: 0.00